

Activity 4: Toolchain and Quality Gates

Course: AI Vibe Coding for iOS Mobile Apps Development (TGS-2023037545)

Mapping: K2, A2

Estimated time: 65 minutes

Objective

Turn organisational design practices into compiler, formatter and test gates.

Deliverable

Warning-clean build, quality script and evidence log

Tools

CMake, compiler warnings, sanitizers, shell

Included files

- ``starter/verify.sh``
- ``starter/quality-gates.txt``
- ``solution/CMakeLists-snippet.txt``
- ``solution/evidence-log.txt``

Before you begin

- Work only with the supplied mock data. Do not add credentials, personal data or signing files.
- Keep the portable C++ core free of Swift, Objective-C and Apple framework dependencies.
- Record the initial state before editing so that verification shows a meaningful change.
- If using an AI coding assistant, treat its output as a proposal and inspect the resulting diff.

Procedure

1. Run ``cd activities/04-toolchain-quality-gates`` and create ``workspace/``. Copy the completed Activity 2 project into it with ``cp -R ../02-portable-cpp-domain-core/solution workspace/domain``.
2. Inspect ``workspace/domain/CMakeLists.txt`` and compare it with ``solution/CMakeLists-snippet.txt``. Confirm C++20 is required and warnings are enabled only for supported compiler families.
3. Merge the strict warning block into the working CMake file. Keep ``-Wall -Wextra -Werror``; do not silence a diagnostic globally to make the build pass.
4. Copy ``starter/verify.sh`` to ``workspace/verify.sh`` and run it explicitly with ``sh workspace/verify.sh``. Expect configure, build and CTest to complete with exit status zero when CMake is available.
5. When CMake is unavailable, verify the equivalent gate with ``c++ -std=c++20 -Wall -Wextra -Werror -I workspace/domain/include workspace/domain/src/BudgetPolicy.cpp``

```
workspace/domain/tests/BudgetPolicyTests.cpp -o workspace/DomainCoreTests &&  
workspace/DomainCoreTests`.
```

6. Inject a controlled defect by adding ``int unused = 1;`` inside ``BudgetPolicy::evaluate``. Re-run the strict build and confirm the unused-variable diagnostic makes the command fail.
7. Copy the failing diagnostic and command into ``workspace/evidence-log.txt``; record the source file and line reported by the compiler.
8. Remove the injected variable at its causal line. Do not weaken ``-Werror`` and do not add an unused suppression.
9. Run the complete gate again and record the successful exit status. Compare your evidence log with ``solution/evidence-log.txt`` for the required Red/Repair/Green structure.
10. Optional when supported: configure with ``cmake -S workspace/domain -B workspace/asan-build -DENABLE_ASAN=ON``, build, and run tests. Record whether the platform supports the sanitizer configuration.
11. Run ``find workspace -maxdepth 2 -type f | sort`` and identify generated binaries or cache data separately from source and verification scripts.
12. Cleanup only ``workspace/domain/build``, ``workspace/asan-build`` and compiled binaries. Retain the strict configuration, verification script and evidence log as A2 evidence.

Verification

A clean checkout can run one command and receive a non-zero exit on the injected defect and zero after repair.

Capture the command output, test result, matrix or screenshot named by the deliverable. A statement such as “it works” is not sufficient evidence.

Troubleshooting

If Apple Clang rejects a flag, gate it by compiler ID and document the equivalent.

Check the earliest failing layer first: portable C++ domain test, then Objective-C++ contract, then Swift state and finally the visible interface.

Cleanup

Remove generated build and sanitizer output, not the verification script.

Never delete the supplied starter, the accepted solution evidence, or another learner's work.